

CMPE 660 Laboratory Exercise 1

USB/PS2 Keyboard Interface

Diego Diaz Peon Gonzalez Pacheco
Performed: 9/7/2026
Submitted: 9/21/2026

TA: Anthony Doan
Drew Ryczek
Dominik Ramut

Lecture Section: 1
Professor: Dr. Lukowiak

By submitting this report, you attest that you neither have given nor have received any assistance (including writing, collecting data, plotting figures, tables or graphs, or using previous student reports as a reference), and you further acknowledge that giving or receiving such assistance will result in a failing grade for this course.

Your Signature: *Diego Diaz Peon Gonzalez Pacheco*

Abstract

In this exercise, a basic hardware interface for receiving data from a USB/PS2 keyboard was implemented on a Nexys4 DDR FPGA, and different ways of implementing VHDL state machines were examined. The exercise was split into four parts, three of which were different implementations of the keyboard interface. One-hot state encoding was used for all of them. In the first implementation, a complex state machine was used where each received bit had its own state, so the bit count was built into the state machine itself. In the second implementation, a simple state machine was used together with an explicit counter and comparator in the datapath. Since one-hot encoding uses one flip-flop per state, having fewer states lowered the register usage: **the first implementation used 40 flip-flops, while the second used only 37.** The third implementation used the same state machine as the first, but instead of the PS2 clock, the FPGA's 100 MHz clock was used together with a divide-by-256 clock divider. The PS2 clock was then used as a sampling enable, and four-stage serial sampling registers were added to the PS2 clock and data lines to sync them to the FPGA clock. Aside from these three implementations, an Integrated Logic Analyzer (ILA) was set up using the given behavioral model, with a trigger set on the value `x"F0"` in the shift register so a full break code could be captured. All three implementations were simulated and also demonstrated on the FPGA.

1 Design Methodology

1.1 PS2 Protocol

Each PS2 transmission is an 11-bit frame made up of a start bit (0), eight data bits sent least significant bit first, an odd parity bit, and a stop bit (1). The PS2 clock only toggles while data is being sent, and it idles high otherwise. An example frame is shown in Figure 1.

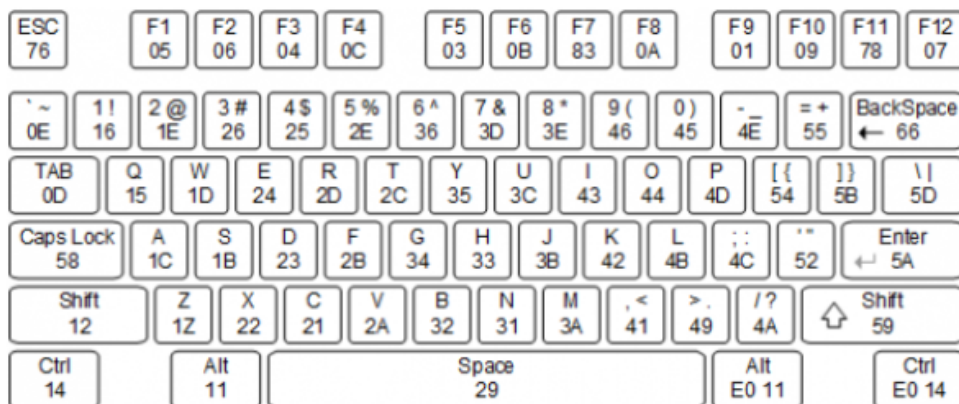


Figure 1: Example PS2 transmission of scan code 16H (the “1” key), from the lab handout

1.2 Common Design Elements

Even though each part had a different control structure, the three implementations shared a common datapath. An 8 bit shift register, made up of eight flip-flops (FFs), received the incoming PS2 data bits. The shift register was enabled by the state machine while the data bits were being read. The outputs of the eight FFs were connected to a chain of XOR gates followed by an inverter, which calculated the odd parity bit.

To display both the current and previous scan codes, three two stage registers were used, each holding a current and a previous value. The first held the received byte, the second held the calculated parity bit, and the third held the received parity bit, which was first loaded from ps2_data into its own register. The current outputs of these registers were concatenated into one bus and the previous outputs into another. Both buses were connected to a 2:1 multiplexer that outputs to the FPGA's LEDs, and the select line of the multiplexer came from an FPGA switch.

Aside from the datapath, each implementation needed a constraints (XDC) file. This file maps every top-level input and output to a pin on the FPGA. For example, the asynchronous reset `BTN_UP` was mapped to the up push button on the board.

1.3 Part 1: Complex State Machine

For the first implementation, skeleton code and a description of the functionality were given, with most of the state machine and datapath already finished. The remaining task was to add the parity checking circuit. Before any code was written, an RTL schematic was created to have a clear picture of the hardware needed. This schematic is shown in Figure 2.

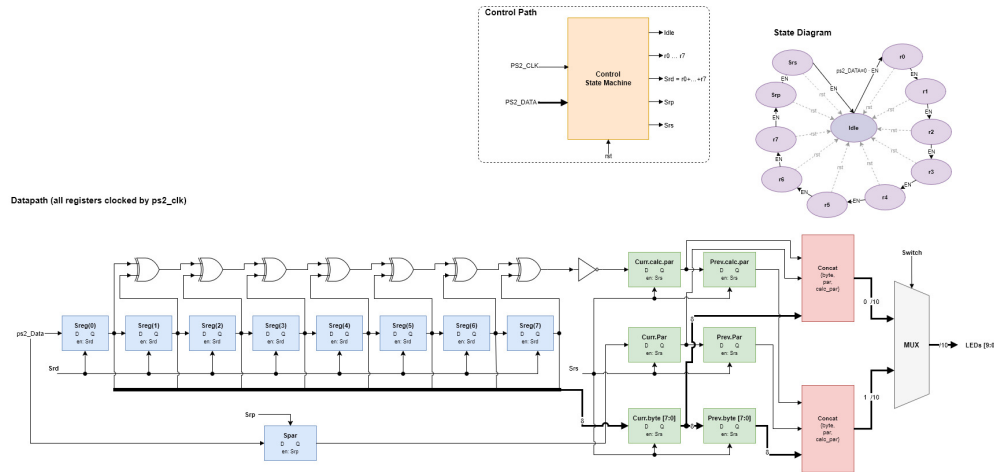


Figure 2: RTL schematic for the complex state machine implementation

In this implementation, each bit of the PS2 frame had its own state, so the bit count was built into the state machine. This caused the state machine to have the states `Idle`, `r0` through `r7`, `rp`, and `rs`. Since the circuit ran in the PS2 clock domain, each state lasted one PS2 clock cycle. `Idle` is the default state, where the machine waited for `ps2_data` to

go low, which signals the start bit. It then moved through **r0**–**r7**, capturing one data bit in each state, until all eight data bits were received. In **rp**, the sent parity bit was captured, and in **rs**, the stop bit was received. The state then changed back to **Idle** to wait for the next frame. Each state corresponds to one bit of the frame in Figure 1, and the full state diagram is shown in Figure 3.

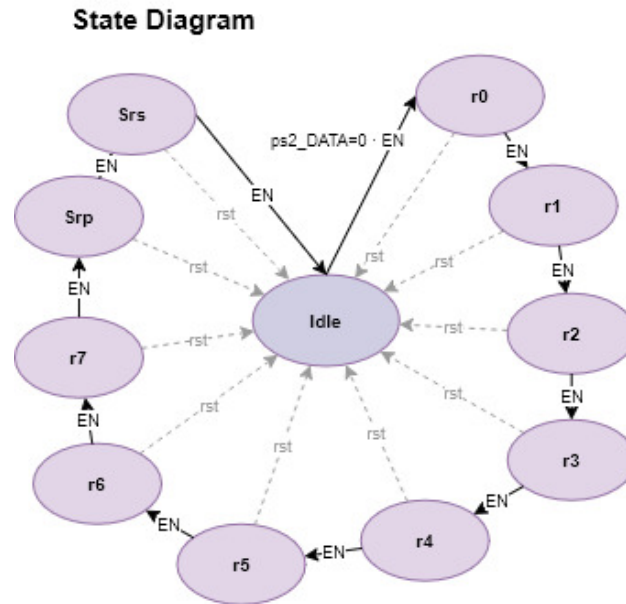


Figure 3: State diagram for the complex state machine implementation

The parity checking circuit described in Section 1.2 was added to the datapath to calculate the expected odd parity bit from the eight received data bits. The calculated and received parity bits were displayed next to each other on the LEDs, so if the two did not match, the byte had been corrupted in transit.

1.4 Part 2: Simple State Machine

For the second implementation, the code from Part 1 was edited so that the state machine had as few states as possible, with the bit counting moved into the datapath. As with Part 1, an RTL schematic was created first, shown in Figure 4.

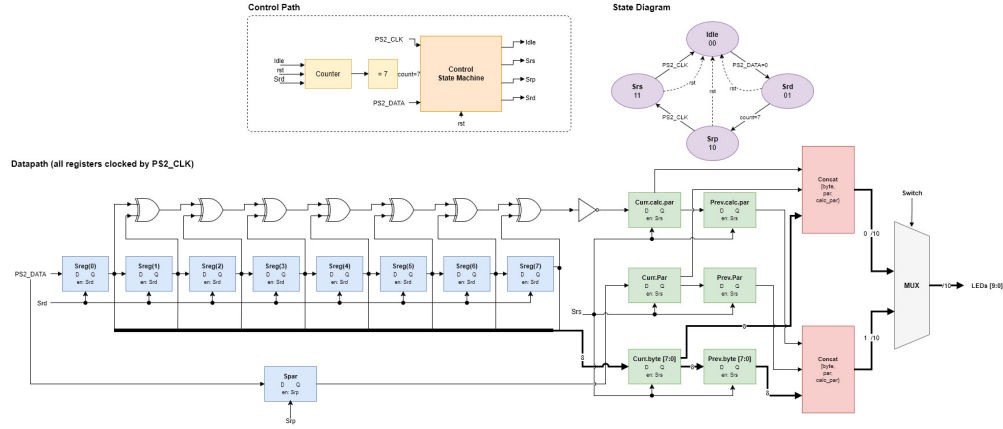


Figure 4: RTL schematic for the simple state machine implementation

The state diagram for this implementation is shown in Figure 5.

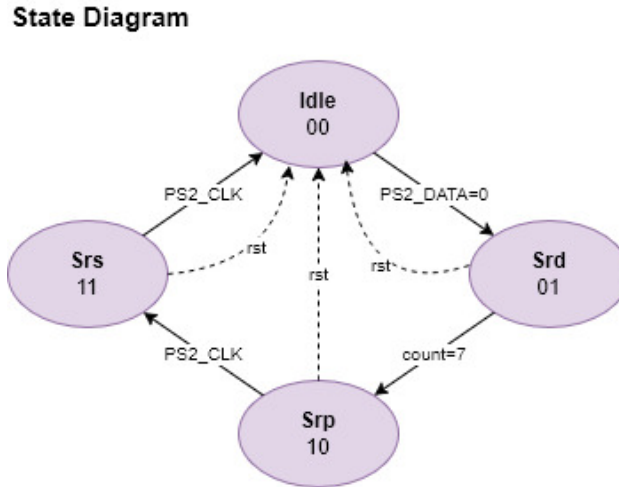


Figure 5: State diagram for the simple state machine implementation

Compared to the Part 1 state machine in Figure 3, the number of states was reduced a lot, down to **idle**, **rd**, **rp**, and **rs**. The eight bit states **r0**–**r7** were replaced by the single read state **rd**. To make up for the fewer states, an explicit 4-bit counter and a comparator were added to the datapath, as shown in Figure 4, with the counter written as its own VHDL process. While the state machine was in **rd**, the counter increased every PS2 clock cycle as each data bit was shifted in. Once the comparator detected that all eight data bits had been received, it signaled the state machine to move to **rp**. The counter then held its value until it was cleared at the start of the next frame. This way, the counter and comparator did the same job as the “extra” states **r0**–**r7** in Part 1. Aside from the added counter and comparator, the datapath was the same as in Part 1.

1.5 Part 3: Integrated Logic Analyzer (ILA)

Part 3 was somewhat different from the other parts, since no new implementation was designed. Instead, it was an introduction to debugging with the Integrated Logic Analyzer (ILA), following the provided tutorial. The given behavioral model, `PS2_behv.vhd`, was used as the design being tested.

Since the ILA needs a free-running clock to sample with, and the PS2 clock only toggles during a transmission, a sample clock had to be made from the board's 100 MHz clock. A Xilinx Clocking Wizard IP core was generated to create a 25 MHz clock. It was added to the design using the provided instantiation template, with the 100 MHz clock connected to `clk_in1` and a new signal connected to `clk_out1`.

The signals to be monitored, `PS2_CLK`, `PS2_DATA`, and `sreg`, were flagged for debugging with the `mark_debug` attribute. After synthesis, the Set Up Debug wizard was used to add the ILA core. Since the behavioral design was not synced to a system clock, the marked signals had no clock domain, so all of them were set to the 25 MHz output of the Clocking Wizard. A sample data depth of 65,536 was selected with capture control enabled. The debug hub was then set up for the slower clock by enabling `C_ENABLE_CLK_DIVIDER` and setting `C_CLK_INPUT_FREQ_HZ` to 25,000,000.

When the bitstream was generated, Vivado also created a debug probes file (`.ltx`), which lets the Hardware Manager find and display the ILA probes once the FPGA is programmed. A trigger was then set to go off whenever `sreg` was equal to `x"F0"`, the first byte of a break code. The trigger position was set to 20,000 samples so that the data and clock activity before the trigger would also be recorded. The core was then armed, and a key was pressed and released to create a break code.

1.6 Part 4: Separate Clock Domains

Part 4 used the same control and datapath structure as Part 1, but the design was moved out of the PS2 clock domain. This implementation used the FPGA's 100 MHz clock for all registers. The clock divider from `cnt_clock_divider.vhd` was used to create an enable pulse, `sclk_enable`, once every 256 clock cycles, which slowed the effective rate down to about 390.6 kHz. As in Parts 1 and 2, an RTL schematic was created first, shown in Figure 6.

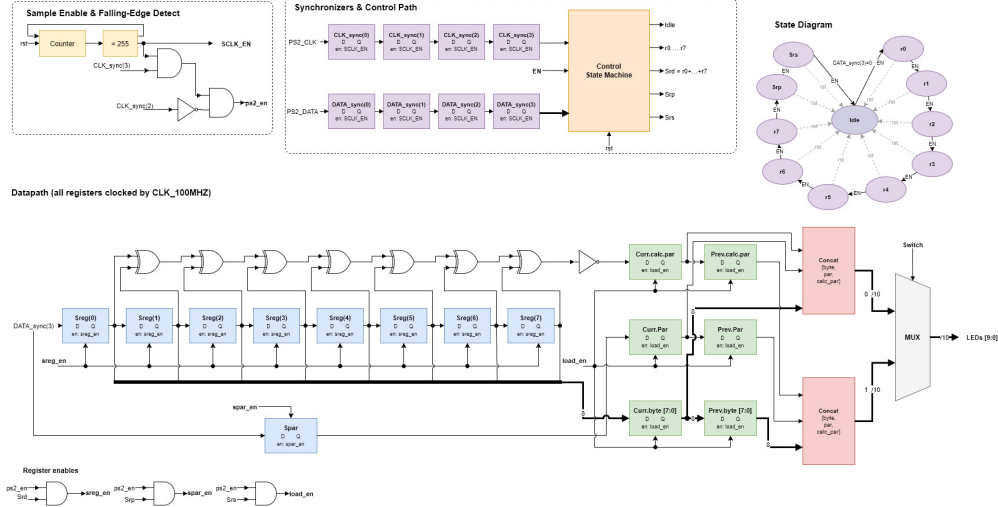


Figure 6: RTL schematic for the separate clock domains implementation

The states and transitions were the same as in Part 1 (Figure 3). What changed was how the design handled the PS2 signals. Since the PS2 clock and data lines are not synced to the FPGA clock, sampling them directly could cause metastability. To sync them, each line was passed through its own four-stage serial sampling register, which shifted in a new sample on every `sclk_enable` pulse. This way, the PS2 clock no longer drove any flip-flops. Instead, its synced samples were used to create an enable signal, `ps2_en`, which was high for one cycle whenever a falling edge was detected in the sampled PS2 clock. This enable took the place of the PS2 clock edge from Part 1, so the state machine only moved to the next state, and the delayed data sample was only shifted in, when `ps2_en` was high. In the datapath, `ps2_en` was ANDed with the state signals to enable or disable the original shift registers.

2 Results and Analysis

2.1 Simulation Results

Each implementation was tested with a testbench that acted as a PS2 keyboard by driving the PS2 clock and data lines with full 11-bit frames. The testbench sent three frames in a row: the make code `x"15"` (the “Q” key), the break code byte `x"F0"`, and `x"15"` again, which is what the keyboard sends for a key press followed by a release. The expected odd parity bit was 0 for `x"15"`, which has three 1s, and 1 for `x"F0"`, which has four. The 10-bit LED output held the selected byte in the lower eight bits and its received and calculated parity bits in the upper two, so the expected outputs were `x"015"` for `x"15"` and `x"3F0"` for `x"F0"`. Near the end of each simulation, the switch was flipped to check that the previous byte could be displayed.

2.1.1 Part 1: Complex State Machine

The simulation of the complex state machine implementation is shown in Figure 7. For each frame, the state machine went through `Idle`, `r0-r7`, `rp`, and `rs`, moving one state every

PS2 clock cycle. The current byte register changed from `x"00"` to `x"15"`, `x"F0"`, and `x"15"` after each stop bit, and the previous byte register followed one frame behind. The LEDs showed `x"015"` and then `x"3F0"`, which shows that the received and calculated parity bits matched for both bytes. When the switch was flipped, the output changed from the current byte (`x"015"`) to the previous byte (`x"3F0"`), which confirmed that both the byte storage and the output multiplexer worked.

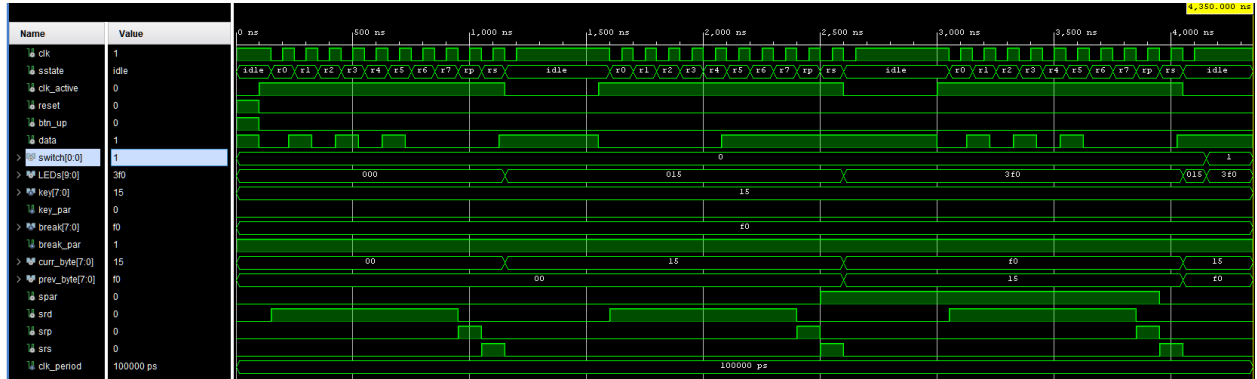


Figure 7: Behavioral simulation of the complex state machine implementation

2.1.2 Part 2: Simple State Machine

The simulation of the simple state machine implementation is shown in Figure 8. In this version, the state machine only went through `idle`, `rd`, `rp`, and `rs` for each frame. While in `rd`, the counter increased once every PS2 clock cycle as each bit was shifted into `sreg`. Since the data is sent least significant bit first, `sreg` shifted right, going through `x"80"`, `x"40"`, `x"A0"`, ..., `x"2A"` before ending at `x"15"`. After the eighth bit, the counter reached 8 and the state changed to `rp`. The counter then held its value until it was cleared at the start of the next frame. The current byte, previous byte, and LED outputs were the same as in Part 1, which confirmed that the counter and comparator did the same job as the per-bit states.

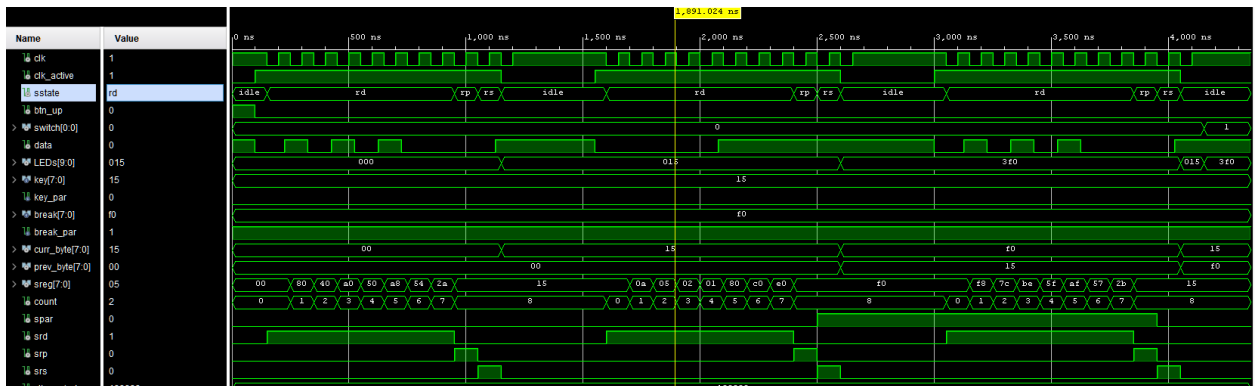


Figure 8: Behavioral simulation of the simple state machine implementation

2.1.3 Part 3: ILA Capture

The ILA capture is shown in Figure 9. Before the trigger, `sreg` held `x"2B"`, the make code of the “F” key, from when the key was pressed. When the key was released, the break code was sent. `sreg` shifted through `x"15"`, `x"0A"`, ..., `x"E0"` as each bit came in, and the trigger went off when it reached `x"F0"`. The second byte of the break code was then shifted in, bringing `sreg` back to `x"2B"`. This means the full break code, `x"F0"` followed by `x"2B"`, was captured in one window. The PS2 clock probe also showed the expected burst of clock pulses for each frame, with the clock idling high in between.

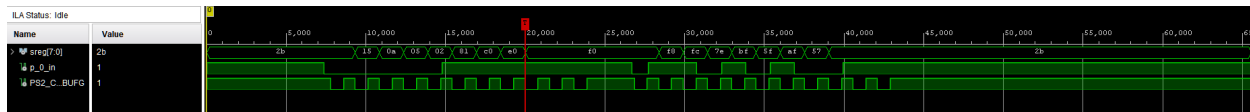


Figure 9: ILA capture of the break code for the “F” key

2.1.4 Part 4: Separate Clock Domains

The simulation of the separate clock domain implementation is shown in Figure 10. Unlike Parts 1 and 2, which were simulated with a 100 ns PS2 clock period to keep the simulation short, this simulation used a realistic PS2 clock period of 100 μ s. This was needed because this design depends on the ratio between the PS2 clock and the 100 MHz FPGA clock. The state sequence, shift register values, and LED outputs were the same as in Part 1.

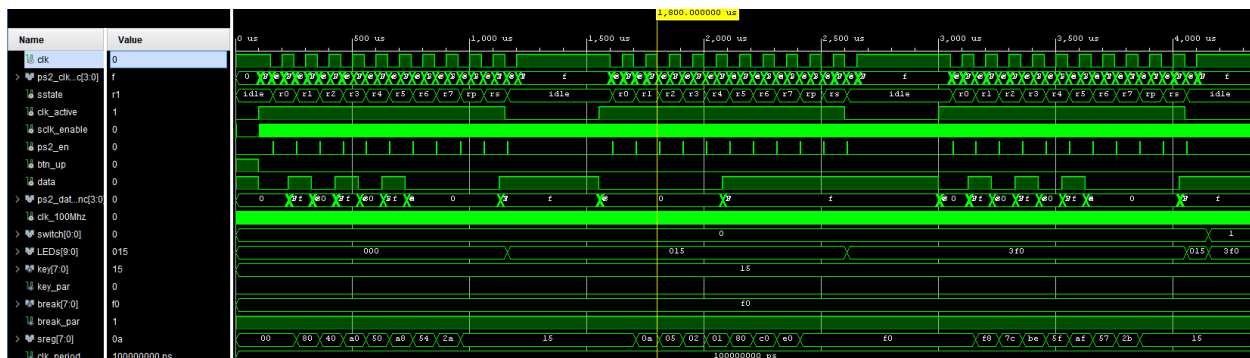


Figure 10: Behavioral simulation of the separate clock domain implementation

A zoomed in view of one PS2 clock edge is shown in Figure 11. The clock divider created a one-cycle `sclk_enable` pulse every 256 clock cycles, or about every 2.56 μ s. On each pulse, a new sample of the PS2 clock was shifted into the four-stage sampling register, which went through `x"0"`, `x"1"`, `x"3"`, `x"7"`, and `x"F"` as the rising edge moved through it. This means the synchronizer added a delay of about four enable periods, around 10 μ s, between a PS2 clock edge and when it was detected. This delay is small compared to the 50 μ s half-period of the PS2 clock, so the delayed data sample was still stable when `ps2_en` went high, and each bit was captured correctly. Also, at one sample every 2.56 μ s, each PS2 line was sampled more than 23 times per PS2 clock period, even at the maximum PS2 frequency of 16.7 kHz, so no edges could be missed.

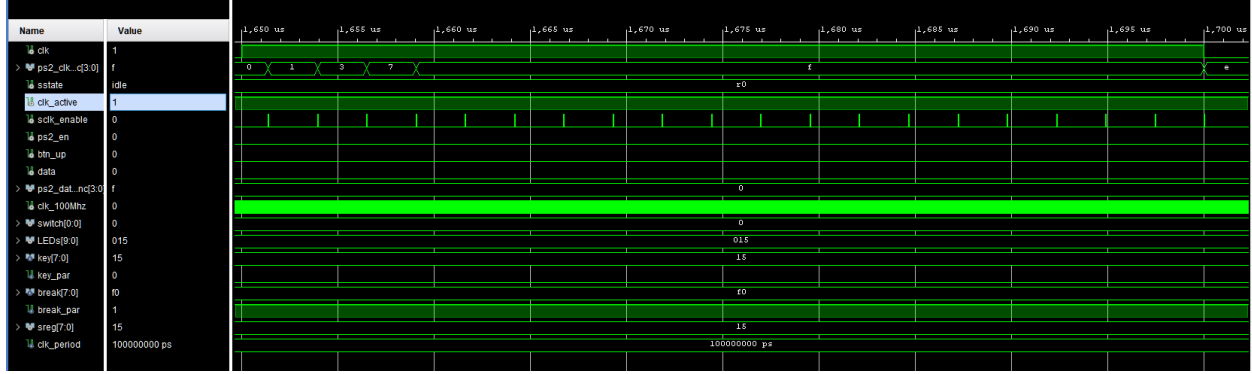


Figure 11: Zoomed in view of the PS2 clock moving through the four-stage sampling register

2.2 Hardware Demonstration

All three implementations were programmed onto the Nexys4 DDR and demonstrated. For each design, pressing a key showed its make code and parity bits on the LEDs. Releasing the key showed the second byte of the break code as the current byte, and `x"F0"` could be seen as the previous byte by flipping the switch. The received and calculated parity bits matched for every key tested, and the asynchronous reset on `BTN_UP` cleared the stored values in all three designs.

2.3 Synthesis Results (Question 1)

The resource utilization reported by Vivado for each implementation is summarized in Table 1. The inferred components from the RTL Component Statistics are listed in Table 2, and the expected flip-flop count for each design, based on its RTL diagram, is broken down in Table 3.

Table 1: Resource utilization of each implementation

Resource	Behavioral	Part 1	Part 2	Part 4
Slice LUTs	19	11	12	21
Flip-flops	29	40	37	57
Latches	0	0	0	0

Table 2: Inferred hardware components

Component	Size	Behavioral	Part 1	Part 2	Part 4
Adders	2-input, 4-bit	1	–	1	–
	2-input, 8-bit	–	–	–	1
XORs	8-input, 1-bit	–	1	1	1
Registers	16-bit	1	–	–	–
	8-bit	1	3	3	4
	4-bit	1	–	1	2
	1-bit	1	5	5	6
Muxes	2-input, 10-bit	–	1	1	1
	2-input, 8-bit	3	–	–	–
	2-input, 4-bit	2	–	1	–
	2-input, 1-bit	2	–	2	1
	4-input, 1-bit	–	1	2	1
	11-input, 1-bit	–	1	–	1

Table 3: Expected flip-flop count by component

Component	Part 1	Part 2	Part 4
Input shift register (sreg)	8	8	8
Current and previous byte registers	16	16	16
Received parity register	1	1	1
Current and previous parity registers	4	4	4
One-hot state register	11	4	11
Bit counter	–	4	–
PS2 clock and data sampling registers	–	–	8
Clock divider counter	–	–	8
Enable register	–	–	1
Total	40	37	57

The flip-flop counts from Vivado matched the expected counts from the RTL diagrams for all three implementations. All three designs used the same 29 flip-flop datapath, so the only differences came from the state machine and the added synchronization hardware.

Part 2 used three fewer flip-flops than Part 1. Since one-hot encoding uses one flip-flop per state, replacing states **r0–r7** with the single state **rd** brought the state register down from 11 flip-flops to 4. However, the new counter added 4 flip-flops back, so the total went down by only three. The counter needed 4 bits instead of 3 because it counted up to 8, as seen in Figure 8. This caused the LUT count to go up slightly, from 11 to 12, since the counter and comparator needed more logic than Part 1’s state machine, where each state just moves to the next one.

Part 4 used the most resources, with 57 flip-flops and 21 LUTs. Compared to Part 1, it added 17 flip-flops: 8 for the two sampling registers, 8 for the divide-by-256 counter ($2^8 = 256$), and 1 for the enable register. The extra LUTs came from the divider’s counter, the edge detection on the PS2 clock, and ANDing **ps2_en** with the datapath enables. Even though

this design used about twice the logic of Part 1, it was the only one that ran fully on the FPGA clock, which protects it from metastability caused by the PS2 inputs.

The components in Table 2 also matched the RTL diagrams. Vivado lists the state machine flip-flops separately, so the registers in this table only include the datapath. In Part 1, the three 8-bit registers were the shift register and the current and previous byte registers. The five 1-bit registers held the received parity bit and the current and previous parity bits. These add up to 29 flip-flops, and with the 11 state flip-flops, that gives the reported 40. As expected, Part 1 had no adder since the bit counting was done by the states. The 8-input XOR was the parity calculation, the 2-input, 10-bit mux was the LED output select, and the 11-input mux came from the 11-state machine.

Part 2 had the same datapath plus the counter, which showed up as a 4-bit adder, a 4-bit register, and a 2-input, 4-bit mux that chooses between counting up and clearing. This gives 33 flip-flops, plus 4 for the states, for the reported 37. The 11-input mux from Part 1 was replaced by 4-input muxes since there were fewer states. The comparator did not show up as its own component because Vivado built it into the LUTs, which explains the extra LUT compared to Part 1.

Part 4 had the Part 1 datapath plus the synchronization hardware. The two 4-bit registers were the PS2 clock and data sampling registers. The extra 8-bit register and the 8-bit adder made up the clock divider, and the extra 1-bit register held the enable. This gives 46 flip-flops, plus 11 for the states, for the reported 57. The 11-input mux from Part 1 also showed up again, which confirms the state machine did not change.

The behavioral model was set up differently from the other three. Since it did not have a separate control unit and datapath, Vivado did not extract a state machine from it. Instead, a single 1-bit register, `vreading`, kept track of whether a frame was being read. All 29 of its flip-flops came from its registers: a 16-bit register holding both bytes, the 8-bit shift register, a 4-bit counter with its adder, and the `vreading` register. No XOR was found in the behavioral model because it did not check parity at all. The counter let the parity bit pass without storing it, and the LED output was only 8 bits wide. This means the behavioral model is not a direct comparison to the other three designs, since part of the difference in flip-flops comes from it not having the five parity registers. It still used 19 LUTs, more than Parts 1 and 2, since all of its logic was written in one process instead of being split up.

No latches were found in any design, which confirms that all the combinational processes were fully specified.

3 Conclusion

In this exercise, a hardware interface for receiving scan codes from a USB/PS2 keyboard was designed, simulated, and implemented on the Nexys4 DDR FPGA in three different ways, and an ILA was used to observe the interface running on the hardware. All three implementations correctly received each 11-bit PS2 frame, stored the current and previous bytes, checked the odd parity bit, and displayed the results on the LEDs through a switch-controlled multiplexer. They behaved the same both in simulation and on the FPGA.

Comparing the implementations showed how splitting the work between the control unit and the datapath affects the hardware used. Moving the bit counting out of the state machine

and into an explicit counter brought the flip-flop count down from 40 in Part 1 to 37 in Part 2. Since one-hot encoding uses one flip-flop per state, the savings came from going from 11 states to 4, and part of it was taken back by the 4-bit counter, while the counter and comparator added a small amount of logic. Syncing the design to the FPGA clock in Part 4 was the most expensive option, using 57 flip-flops and 21 LUTs. However, it was also the only design that did not use the PS2 clock as a clock, which makes it the most protected against metastability and the best approach for a bigger system. In every case, the flip-flop counts and inferred components reported by Vivado matched what was expected from the RTL diagrams, which confirms that the VHDL described the intended hardware.

The ILA part of the exercise showed how useful on-chip debugging can be. By triggering on the F0 byte and storing samples before and after the trigger, a full break code was captured straight from the running hardware. This gave a view of internal signals that the LEDs alone could not show, and it worked well alongside the testbench simulations.

Exercise 1: USB/PS2 Keyboard Interface

Student's Name: Diego Diaz Section: _____

Demo		Point Value	Points Earned	Date
Part 1: Complex	Hardware Demo	10	10	AD 9/21
	RTL Diagram	10	10	
Part 2: Simple	Simulation	10	10	
	Hardware Demo	10	10	
Part 3: ILA	Hardware Demo	10	10	
Part 4: Clock Crossing	RTL Diagram	10	10	
	Simulation	10	10	
	Hardware Demo	10	10	

To receive any grading credit students must earn points for both the demonstration and the report.

Report		Point Value	Points Earned	Comments
Report	Discussion	10		
	Question	10		
Total for demo and report		100		

An electronic submission of project files via MyCourses is required. For information on how to properly submit the files, see the document "MyCoursesTut.pdf" on MyCourses.

Required file structure of tarball submission for this exercise:

```

Ex1
├── Part2
│   ├── [all necessary .vhd files]
│   ├── Part2.xdc
│   └── Part2.tcl
├── Part3
│   ├── [all necessary .vhd files]
│   ├── Part3.xdc
│   └── Part3.tcl
└── Part4
    ├── [all necessary .vhd files]
    ├── Part4.xdc
    └── Part4.tcl
    
```

Figure 12: Demo sign-off sheet